

Beginner's Guide to Data Analysis & Visualization in RStudio

A Step-by-Step Project Using the Student Performance Dataset

Instructional Document

No prior programming or statistics experience required

Dataset file: student_performance.csv

1. Introduction

This guide walks you through a complete, beginner-friendly data analysis and visualization project using RStudio. You do not need any prior experience with R, statistics, or programming — every step is explained in detail, including where to click and what each line of code does.

By the end of this project you will be able to:

- Install and set up RStudio on your computer
- Import a CSV dataset into R
- Explore and understand a dataset's structure
- Clean and summarize data
- Create meaningful visualizations using ggplot2 (bar charts, histograms, boxplots, and scatter plots)
- Draw simple, data-driven conclusions from your analysis

Dataset used in this project

student_performance.csv — a dataset of 200 students containing study habits, attendance, sleep, parental education, and their final exam scores. It is provided alongside this document.

2. Installing R and RStudio

R is the programming language; RStudio is the application (IDE) you use to write and run R code. You need to install both, in this order.

Step 1: Install R

1. Go to <https://cran.r-project.org>
2. Click the download link for your operating system (Windows / macOS / Linux).
3. Run the installer and accept all default options.

Step 2: Install RStudio Desktop

1. Go to <https://posit.co/download/rstudio-desktop/>
2. Download the free RStudio Desktop version for your operating system.
3. Run the installer and accept all default options.
4. Open RStudio. You should see four panels: Source (top-left), Console (bottom-left), Environment/History (top-right), and Files/Plots/Packages/Help (bottom-right).

Tip

If you only see three panels, click File > New File > R Script to open the Source panel — that is where you will write your code.

3. Setting Up Your Project

Step 1: Create a new RStudio Project

1. In RStudio, go to File > New Project > New Directory > New Project.
2. Name it StudentPerformanceAnalysis and choose a folder on your computer to save it in.
3. Click Create Project. RStudio will reopen inside this new project folder.

Step 2: Add the dataset

1. Copy the file student_performance.csv into the same project folder you just created.
2. In the bottom-right panel, click the Files tab and confirm you can see student_performance.csv listed there.

Step 3: Create your R script

1. Go to File > New File > R Script.
2. Save it immediately as analysis.R (Ctrl/Cmd + S) inside your project folder.

You will write all the code from this guide into analysis.R. Run each line by placing your cursor on it and pressing Ctrl+Enter (Windows/Linux) or Cmd+Enter (Mac).

4. Installing and Loading Required Packages

R's base functions can do a lot, but the tidyverse collection of packages makes data analysis and visualization much easier. We will mainly use two packages from it: dplyr (data handling) and ggplot2 (visualization).

Step 1: Install the packages (only needs to be done once)

```
install.packages("tidyverse")
```

This may take a minute or two. You will see progress messages in the Console.

Step 2: Load the packages (do this every time you start a new session)

```
library(tidyverse) # loads ggplot2, dplyr, readr, and more
```

5. Importing the Dataset

```
students <- read_csv("student_performance.csv")
```

This reads the CSV file and stores it as a data frame named students. A data frame is R's table-like structure — think of it as a spreadsheet inside R, with rows as observations (students) and columns as variables (attributes).

Check that it loaded correctly

```
View(students) # opens a spreadsheet-style viewer in a new tab
```

```
head(students)      # shows the first 6 rows
dim(students)       # shows number of rows and columns
```

6. Exploring the Dataset

Before visualizing anything, always get familiar with your data. This step is called exploratory data analysis (EDA).

6.1 Structure of the data

```
str(students)      # shows column names, data types, and sample values
```

Notice that columns like Gender, PartTimeJob, and Result are text (character) columns, while StudyHoursPerWeek and FinalExamScore are numeric.

6.2 Summary statistics

```
summary(students)  # min, max, mean, median for every numeric column
```

This instantly tells you the range and average of each numeric variable — for example, the average FinalExamScore and how spread out the scores are.

6.3 Checking for missing values

```
colSums(is.na(students))  # counts missing (NA) values per column
```

This dataset has no missing values, but it is good practice to always check, since real-world data often does.

6.4 Counting categories

```
table(students$Gender)
table(students$Result)
table(students$PartTimeJob)
```

The \$ symbol accesses a single column from the data frame — students\$Gender means “the Gender column of the students data frame.”

7. Basic Data Analysis with dplyr

7.1 Average exam score

```
mean(students$FinalExamScore)
median(students$FinalExamScore)
```

7.2 Correlation between numeric variables

Correlation measures how strongly two numeric variables move together, on a scale from -1 to +1. Values near +1 mean a strong positive relationship; values near 0 mean little to no linear relationship.

```
cor(students$StudyHoursPerWeek, students$FinalExamScore)
cor(students$AttendancePercent, students$FinalExamScore)
cor(students$SleepHoursPerNight, students$FinalExamScore)
```

Run these one at a time and note the values — you will need them for your project report and to interpret the scatter plot in Section 8.4.

8. Data Visualization with ggplot2

ggplot2 builds plots in layers. Every plot follows the same basic recipe:

```
ggplot(data = <dataset>, aes(x = <variable>)) +  
  geom_<type>() +  
  labs(title = "...", x = "...", y = "...")
```

ggplot() sets up the plot and which dataset/variables to use, aes() maps variables to visual properties (x-axis, y-axis, color, etc.), geom_...() adds the actual visual layer (bars, points, lines), and labs() adds titles and axis labels. Layers are joined with a + sign.

8.1 Histogram — Distribution of Final Exam Scores

A histogram shows how a single numeric variable is distributed across ranges of values.

```
ggplot(students, aes(x = FinalExamScore)) +  
  geom_histogram(binwidth = 5, fill = "steelblue", color = "white") +  
  labs(title = "Distribution of Final Exam Scores",  
        x = "Final Exam Score", y = "Number of Students")
```

What to look for: is the distribution roughly bell-shaped (normal)? Is it skewed toward high or low scores? Are there any unusual gaps?

8.2 Bar Chart — Pass vs Fail Count

Bar charts are used to compare counts or values across categories.

```
ggplot(students, aes(x = Result, fill = Result)) +  
  geom_bar() +  
  labs(title = "Number of Students: Pass vs Fail",  
        x = "Result", y = "Number of Students")
```

8.3 Boxplot — Exam Score by Part-Time Job Status

A boxplot summarizes a numeric variable's distribution (median, quartiles, and outliers) across categories — useful for comparing groups at a glance.

```
ggplot(students, aes(x = PartTimeJob, y = FinalExamScore, fill = PartTimeJob)) +  
  geom_boxplot() +  
  labs(title = "Final Exam Score by Part-Time Job Status",  
        x = "Has a Part-Time Job?", y = "Final Exam Score")
```

The thick line inside each box is the median. The box spans the middle 50% of scores. Points beyond the whiskers are potential outliers.

8.4 Scatter Plot — Study Hours vs Final Exam Score

Scatter plots reveal relationships between two numeric variables. Adding a trend line makes the relationship easier to see.

```
ggplot(students, aes(x = StudyHoursPerWeek, y = FinalExamScore)) +
  geom_point(color = "darkorange", alpha = 0.7) +
  geom_smooth(method = "lm", se = FALSE, color = "black") +
  labs(title = "Study Hours vs Final Exam Score",
        x = "Study Hours per Week", y = "Final Exam Score")
```

`geom_smooth(method = "lm")` adds a straight best-fit trend line. An upward-sloping line confirms the positive correlation you calculated in Section 7.3.

9. Saving Your Plots

To save your most recent plot as an image file, use `ggsave()` right after creating it:

```
ggsave("study_hours_vs_score.png", width = 7, height = 5, dpi = 300)
```

Saved images will appear in your project folder and can be inserted into your project report.

10. Suggested Exercises (Practice on Your Own)

- Create a boxplot comparing `FinalExamScore` across `ExtraClasses` (Yes/No).
- Create a scatter plot of `AttendancePercent` vs `FinalExamScore` with a trend line.
- Use `group_by()` and `summarise()` to compare average `PreviousGPA` across `ParentalEducation` levels.
- Try changing `geom_histogram()`'s `binwidth` to 2 and to 10 — how does the shape of the plot change?
- Color the scatter plot in Section 8.4 by `Gender` using `aes(color = Gender)` and see if the relationship differs between groups.

11. Common Errors and How to Fix Them

Error message	Likely cause & fix
could not find function "ggplot"	You forgot to run <code>library(tidyverse)</code> in this session. Re-run it.
object 'students' not found	You have not run the <code>read_csv()</code> line yet, or you cleared your Environment. Re-run Section 5.
cannot open file 'student_performance.csv'	The CSV is not in your project folder, or your working directory is wrong. Check with <code>getwd()</code> and move the file there.
unexpected symbol / unexpected ','	A syntax typo — usually a missing comma, bracket, or quotation mark. Check the line just above the error too.
Error: Discrete value supplied to continuous scale	You are plotting a text column on an axis meant for numbers, or vice versa. Check the column type with <code>str(students)</code> .

12. Quick Reference — All Code in Order

Copy this block into `analysis.R` to run the entire project from start to finish.

```
# 1. Load packages
library(tidyverse)
```

```
# 2. Import data
students <- read_csv("student_performance.csv")

# 3. Explore data
str(students)
summary(students)
colSums(is.na(students))

# 4. Basic analysis
mean(students$FinalExamScore)
students %>% group_by(Gender) %>% summarise(AvgScore = mean(FinalExamScore))
cor(students$StudyHoursPerWeek, students$FinalExamScore)

# 5. Visualizations
ggplot(students, aes(x = FinalExamScore)) +
  geom_histogram(binwidth = 5, fill = "steelblue", color = "white")

ggplot(students, aes(x = Result, fill = Result)) +
  geom_bar()

ggplot(students, aes(x = PartTimeJob, y = FinalExamScore, fill = PartTimeJob)) +
  geom_boxplot()

ggplot(students, aes(x = StudyHoursPerWeek, y = FinalExamScore)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```